

JavaScript Number Methods

The toString() Method

toString() returns a number as a string.

All number methods can be used on any type of numbers (literals, variables, or expressions):

Example

```
var x = 123;  
x.toString();           // returns 123 from variable x  
(123).toString();      // returns 123 from literal 123  
(100 + 23).toString(); // returns 123 from expression 100 + 23
```

The toExponential() Method

toExponential() returns a string, with a number rounded and written using exponential notation.

A parameter defines the number of characters behind the decimal point:

Example

```
var x = 9.656;  
x.toExponential(2); // returns 9.66e+0  
x.toExponential(4); // returns 9.6560e+0  
x.toExponential(6); // returns 9.656000e+0
```

The parameter is optional. If you don't specify it, JavaScript will not round the number.

The toFixed() Method

toFixed() returns a string, with the number written with a specified number of decimals:

Example

```
var x = 9.656;  
x.toFixed(0);           // returns 10  
x.toFixed(2);           // returns 9.66  
x.toFixed(4);           // returns 9.6560  
x.toFixed(6);           // returns 9.656000
```

The toPrecision() Method

toPrecision() returns a string, with a number written with a specified length:

Example

```
var x = 9.656;  
x.toPrecision();        // returns 9.656  
x.toPrecision(2);       // returns 9.7  
x.toPrecision(4);       // returns 9.656  
x.toPrecision(6);       // returns 9.65600
```

The valueOf() Method

valueOf() returns a number as a number.

Example

```
var x = 123;  
x.valueOf();             // returns 123 from variable x  
(123).valueOf();        // returns 123 from literal 123  
(100 + 23).valueOf();   // returns 123 from expression 100 + 23
```

In JavaScript, a number can be a primitive value (typeof = number) or an object (typeof = object).

The valueOf() method is used internally in JavaScript to convert Number objects to primitive values.

There is no reason to use it in your code.

All JavaScript data types have a `valueOf()` and a `toString()` method.

Converting Variables to Numbers

There are 3 JavaScript methods that can be used to convert variables to numbers:

- The `Number()` method
- The `parseInt()` method
- The `parseFloat()` method

These methods are not **number** methods, but **global** JavaScript methods.

Global JavaScript Methods

JavaScript global methods can be used on all JavaScript data types.

These are the most relevant methods, when working with numbers:

Method	Description
<code>Number()</code>	Returns a number, converted from its argument.
<code>parseFloat()</code>	Parses its argument and returns a floating point number
<code>parseInt()</code>	Parses its argument and returns an integer

The Number() Method

Number() can be used to convert JavaScript variables to numbers:

Example

```
Number(true);           // returns 1
Number(false);          // returns 0
Number("10");           // returns 10
Number(" 10");          // returns 10
Number("10 ");          // returns 10
Number(" 10 ");         // returns 10
Number("10.33");        // returns 10.33
Number("10,33");        // returns NaN
Number("10 33");        // returns NaN
Number("John");         // returns NaN
```

If the number cannot be converted, NaN (Not a Number) is returned.

The Number() Method Used on Dates

Number() can also convert a date to a number:

Example

```
Number(new Date("2017-09-30")); // returns 1506729600000
```

The Number() method above returns the number of milliseconds since 1.1.1970.

The parseInt() Method

parseInt() parses a string and returns a whole number. Spaces are allowed. Only the first number is returned:

Example

```
parseInt("10");           // returns 10
parseInt("10.33");         // returns 10
parseInt("10 20 30");      // returns 10
parseInt("10 years");      // returns 10
parseInt("years 10");      // returns NaN
```

If the number cannot be converted, NaN (Not a Number) is returned.

The parseFloat() Method

parseFloat() parses a string and returns a number. Spaces are allowed. Only the first number is returned:

Example

```
parseFloat("10");          // returns 10
parseFloat("10.33");       // returns 10.33
parseFloat("10 20 30");    // returns 10
parseFloat("10 years");    // returns 10
parseFloat("years 10");    // returns NaN
```

If the number cannot be converted, NaN (Not a Number) is returned.

Number Properties

Property	Description
MAX_VALUE	Returns the largest number possible in JavaScript

MIN_VALUE	Returns the smallest number possible in JavaScript
POSITIVE_INFINITY	Represents infinity (returned on overflow)
NEGATIVE_INFINITY	Represents negative infinity (returned on overflow)
NaN	Represents a "Not-a-Number" value

JavaScript MIN_VALUE and MAX_VALUE

Example

```
var x = Number.MAX_VALUE;
```

Example

```
var x = Number.MIN_VALUE;
```

JavaScript POSITIVE_INFINITY

Example

```
var x = Number.POSITIVE_INFINITY;
```

POSITIVE_INFINITY is returned on overflow:

Example

```
var x = 1 / 0;
```

JavaScript NEGATIVE_INFINITY

Example

```
var x = Number.NEGATIVE_INFINITY;
```

NEGATIVE_INFINITY is returned on overflow:

Example

```
var x = -1 / 0;
```

JavaScript NaN - Not a Number

Example

```
var x = Number.NaN;
```

NaN is a JavaScript reserved word indicating that a number is not a legal number.

Trying to do arithmetic with a non-numeric string will result in NaN (Not a Number):

Example

```
var x = 100 / "Apple"; // x will be NaN (Not a Number)
```

Number Properties Cannot be Used on Variables

Number properties belongs to the JavaScript's number object wrapper called **Number**.

These properties can only be accessed as **Number**.MAX_VALUE.

Using *myNumber*.MAX_VALUE, where *myNumber* is a variable, expression, or value, will return undefined:

Example

```
var x = 6;  
var y = x.MAX_VALUE;    // y becomes undefined
```

Array iteration methods operate on every array item.

Array.forEach()

The `forEach()` method calls a function (a callback function) once for each array element.

Example

```
var txt = "";  
var numbers = [45, 4, 9, 16, 25];  
numbers.forEach(myFunction);  
  
function myFunction(value, index, array) {  
    txt = txt + value + "<br>";  
}
```

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

The example above uses only the value parameter. The example can be rewritten to:

Example

```
var txt = "";  
var numbers = [45, 4, 9, 16, 25];  
numbers.forEach(myFunction);  
  
function myFunction(value) {  
    txt = txt + value + "<br>";  
}
```

Array.forEach() is supported in all browsers except Internet Explorer 8 or earlier:

Array.map()

The map() method creates a new array by performing a function on each array element.

The map() method does not execute the function for array elements without values.

The map() method does not change the original array.

This example multiplies each array value by 2:

Example

```
var numbers1 = [45, 4, 9, 16, 25];
var numbers2 = numbers1.map(myFunction);

function myFunction(value, index, array) {
    return value * 2;
}
```

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

When a callback function uses only the value parameter, the index and array parameters can be omitted:

Example

```
var numbers1 = [45, 4, 9, 16, 25];
var numbers2 = numbers1.map(myFunction);

function myFunction(value) {
    return value * 2;
}
```

Array.map() is supported in all browsers except Internet Explorer 8 or earlier.

Array.filter()

The filter() method creates a new array with array elements that passes a test.

This example creates a new array from elements with a value larger than 18:

Example

```
var numbers = [45, 4, 9, 16, 25];
var over18 = numbers.filter(myFunction);

function myFunction(value, index, array) {
  return value > 18;
}
```

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

In the example above, the callback function does not use the index and array parameters, so they can be omitted:

Example

```
var numbers = [45, 4, 9, 16, 25];
var over18 = numbers.filter(myFunction);

function myFunction(value) {
  return value > 18;
}
```

Array.filter() is supported in all browsers except Internet Explorer 8 or earlier.

Array.reduce()

The reduce() method runs a function on each array element to produce (reduce it to) a single value.

The reduce() method works from left-to-right in the array. See also reduceRight().

The `reduce()` method does not reduce the original array.

This example finds the sum of all numbers in an array:

Example

```
var numbers1 = [45, 4, 9, 16, 25];
var sum = numbers1.reduce(myFunction);

function myFunction(total, value, index, array) {
  return total + value;
}
```

Note that the function takes 4 arguments:

- The total (the initial value / previously returned value)
- The item value
- The item index
- The array itself

The example above does not use the index and array parameters. It can be rewritten to:

Example

```
var numbers1 = [45, 4, 9, 16, 25];
var sum = numbers1.reduce(myFunction);

function myFunction(total, value) {
  return total + value;
}
```

The `reduce()` method can accept an initial value:

Example

```
var numbers1 = [45, 4, 9, 16, 25];
var sum = numbers1.reduce(myFunction, 100);

function myFunction(total, value) {
  return total + value;
}
```

`Array.reduce()` is supported in all browsers except Internet Explorer 8 or earlier.

Array.reduceRight()

The `reduceRight()` method runs a function on each array element to produce (reduce it to) a single value.

The `reduceRight()` works from right-to-left in the array. See also `reduce()`.

The `reduceRight()` method does not reduce the original array.

This example finds the sum of all numbers in an array:

Example

```
var numbers1 = [45, 4, 9, 16, 25];
var sum = numbers1.reduceRight(myFunction);

function myFunction(total, value, index, array) {
    return total + value;
}
```

Note that the function takes 4 arguments:

- The total (the initial value / previously returned value)
- The item value
- The item index
- The array itself

The example above does not use the index and array parameters. It can be rewritten to:

Example

```
var numbers1 = [45, 4, 9, 16, 25];
var sum = numbers1.reduceRight(myFunction);

function myFunction(total, value) {
    return total + value;
}
```

`Array.reduce()` is supported in all browsers except Internet Explorer 8 or earlier.

Array.every()

The every() method check if all array values pass a test.

This example check if all array values are larger than 18:

Example

```
var numbers = [45, 4, 9, 16, 25];
var allOver18 = numbers.every(myFunction);

function myFunction(value, index, array) {
  return value > 18;
}
```

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

When a callback function uses the first parameter only (value), the other parameters can be omitted:

Example

```
var numbers = [45, 4, 9, 16, 25];
var allOver18 = numbers.every(myFunction);

function myFunction(value) {
  return value > 18;
}
```

Array.every() is supported in all browsers except Internet Explorer 8 or earlier.

Array.some()

The some() method check if some array values pass a test.

This example check if some array values are larger than 18:

Example

```
var numbers = [45, 4, 9, 16, 25];
var someOver18 = numbers.some(myFunction);

function myFunction(value, index, array) {
    return value > 18;
}
```

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

Array.some() is supported in all browsers except Internet Explorer 8 or earlier.

Array.indexOf()

Search an array for an element value and returns its position.

Note: The first item has position 0, the second item has position 1, and so on.

Example

Search an array for the item "Apple":

```
var fruits = ["Apple", "Orange", "Apple", "Mango"];
var a = fruits.indexOf("Apple");
```

Array.indexOf() is supported in all browsers except Internet Explorer 8 or earlier.

Syntax

```
array.indexOf(item, start)
```

item Required. The item to search for.

start Optional. Where to start the search. Negative values will start at the given position counted from the end, and search to the end.

Array.indexOf() returns -1 if the item is not found.

If the item is present more than once, it returns the position of the first occurrence.

Array.lastIndexOf()

Array.lastIndexOf() is the same as Array.indexOf(), but searches from the end of the array.

Example

Search an array for the item "Apple":

```
var fruits = ["Apple", "Orange", "Apple", "Mango"];  
var a = fruits.lastIndexOf("Apple");
```

Array.lastIndexOf() is supported in all browsers except Internet Explorer 8 or earlier.

Syntax

array.lastIndexOf(item, start)

item

Required. The item to search for

start

Optional. Where to start the search. Negative values will start at the given position counted from the end, and search to the beginning

Array.find()

The find() method returns the value of the first array element that passes a test function.

This example finds (returns the value of) the first element that is larger than 18:

Example

```
var numbers = [4, 9, 16, 25, 29];
var first = numbers.find(myFunction);

function myFunction(value, index, array) {
  return value > 18;
}
```

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

Array.find() is not supported in older browsers. The first browser versions with full support is listed below.

Array.findIndex()

The findIndex() method returns the index of the first array element that passes a test function.

This example finds the index of the first element that is larger than 18:

Example

```
var numbers = [4, 9, 16, 25, 29];
var first = numbers.findIndex(myFunction);

function myFunction(value, index, array) {
  return value > 18;
}
```

Note that the function takes 3 arguments:

- The item value
- The item index
- The array itself

`Array.findIndex()` is not supported in older browsers. The first browser versions with full support is listed below.

JavaScript **Date Object** lets us work with dates:

Fri Jan 04 2019 15:28:09 GMT+0530 (India Standard Time)

Year: 2019 Month: 1 Day: 4 Hours: 15 Minutes 28 Seconds: 9

Example

```
var d = new Date();
```

JavaScript Date Output

By default, JavaScript will use the browser's time zone and display a date as a full text string:

Fri Jan 04 2019 15:28:09 GMT+0530 (India Standard Time)

You will learn much more about how to display dates, later in this tutorial.

Creating Date Objects

Date objects are created with the **new Date()** constructor.

There are **4 ways** to create a new date object:

```
new Date()  
new Date(year, month, day, hours, minutes, seconds, milliseconds)  
new Date(milliseconds)  
new Date(date string)
```

new Date()

new Date() creates a new date object with the **current date and time**:

Example

```
var d = new Date();
```

Date objects are static. The computer time is ticking, but date objects are not.

new Date(*year, month, ...*)

new Date(*year, month, ...*) creates a new date object with a **specified date and time**.

7 numbers specify year, month, day, hour, minute, second, and millisecond (in that order):

Example

```
var d = new Date(2018, 11, 24, 10, 33, 30, 0);
```

JavaScript counts months from 0 to 11.

January is 0. December is 11.

6 numbers specify year, month, day, hour, minute, second:

Example

```
var d = new Date(2018, 11, 24, 10, 33, 30);
```

5 numbers specify year, month, day, hour, and minute:

Example

```
var d = new Date(2018, 11, 24, 10, 33);
```

4 numbers specify year, month, day, and hour:

Example

```
var d = new Date(2018, 11, 24, 10);
```

3 numbers specify year, month, and day:

Example

```
var d = new Date(2018, 11, 24);
```

2 numbers specify year and month:

Example

```
var d = new Date(2018, 11);
```

You cannot omit month. If you supply only one parameter it will be treated as milliseconds.

Example

```
var d = new Date(2018);
```

Previous Century

One and two digit years will be interpreted as 19xx:

Example

```
var d = new Date(99, 11, 24);
```

Example

```
var d = new Date(9, 11, 24);
```

new Date(*dateString*)

new Date(*dateString*) creates a new date object from a **date string**:

Example

```
var d = new Date("October 13, 2014 11:13:00");
```

Date strings are described in the next chapter.

JavaScript Stores Dates as Milliseconds

JavaScript stores dates as number of milliseconds since January 01, 1970, 00:00:00 UTC (Universal Time Coordinated).

Zero time is January 01, 1970 00:00:00 UTC.

Now the time is: **1546595889712** milliseconds past January 01, 1970

`new Date(milliseconds)`

`new Date(milliseconds)` creates a new date object as **zero time plus milliseconds**:

Example

```
var d = new Date(0);
```

01 January 1970 **plus** 100 000 000 000 milliseconds is approximately 03 March 1973:

Example

```
var d = new Date(100000000000);
```

January 01 1970 **minus** 100 000 000 000 milliseconds is approximately October 31 1966:

Example

```
var d = new Date(-100000000000);
```

Example

```
var d = new Date(86400000);
```

One day (24 hours) is 86 400 000 milliseconds.

Date Methods

When a Date object is created, a number of **methods** allow you to operate on it.

Date methods allow you to get and set the year, month, day, hour, minute, second, and millisecond of date objects, using either local time or UTC (universal, or GMT) time.

Date methods and time zones are covered in the next chapters.

Displaying Dates

JavaScript will (by default) output dates in full text string format:

```
Wed Mar 25 2015 05:30:00 GMT+0530 (India Standard Time)
```

When you display a date object in HTML, it is automatically converted to a string, with the **toString()** method.

Example

```
d = new Date();  
document.getElementById("demo").innerHTML = d;
```

Same as:

```
d = new Date();  
document.getElementById("demo").innerHTML = d.toString();
```

The **toUTCString()** method converts a date to a UTC string (a date display standard).

Example

```
var d = new Date();  
document.getElementById("demo").innerHTML = d.toUTCString();
```

The **toDateString()** method converts a date to a more readable format:

Example

```
var d = new Date();  
document.getElementById("demo").innerHTML = d.toDateString();
```

JavaScript Get Date Methods

These methods can be used for getting information from a date object:

Method	Description
getFullYear()	Get the year as a four digit number (yyyy)
getMonth()	Get the month as a number (0-11)
getDate()	Get the day as a number (1-31)
getHours()	Get the hour (0-23)
getMinutes()	Get the minute (0-59)

<code>getSeconds()</code>	Get the second (0-59)
<code>getMilliseconds()</code>	Get the millisecond (0-999)
<code>getTime()</code>	Get the time (milliseconds since January 1, 1970)
<code>getDay()</code>	Get the weekday as a number (0-6)
<code>Date.now()</code>	Get the time. ECMAScript 5.

The `getTime()` Method

The **`getTime()`** method returns the number of milliseconds since January 1, 1970:

Example

```
var d = new Date();  
document.getElementById("demo").innerHTML = d.getTime();
```

The `getFullYear()` Method

The **`getFullYear()`** method returns the year of a date as a four digit number:

Example

```
var d = new Date();  
document.getElementById("demo").innerHTML = d.getFullYear();
```

The getMonth() Method

The **getMonth()** method returns the month of a date as a number (0-11):

Example

```
var d = new Date();  
document.getElementById("demo").innerHTML = d.getMonth();
```

In JavaScript, the first month (January) is month number 0, so December returns month number 11.

You can use an array of names, and getMonth() to return the month as a name:

Example

```
var d = new Date();  
var months =  
["January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December"];  
document.getElementById("demo").innerHTML = months[d.getMonth()];
```

The getDate() Method

The **getDate()** method returns the day of a date as a number (1-31):

Example

```
var d = new Date();  
document.getElementById("demo").innerHTML = d.getDate();
```

The getHours() Method

The **getHours()** method returns the hours of a date as a number (0-23):

Example

```
var d = new Date();  
document.getElementById("demo").innerHTML = d.getHours();
```

The getMinutes() Method

The **getMinutes()** method returns the minutes of a date as a number (0-59):

Example

```
var d = new Date();  
document.getElementById("demo").innerHTML = d.getMinutes();
```

The getSeconds() Method

The **getSeconds()** method returns the seconds of a date as a number (0-59):

Example

```
var d = new Date();  
document.getElementById("demo").innerHTML = d.getSeconds();
```

The getMilliseconds() Method

The **getMilliseconds()** method returns the milliseconds of a date as a number (0-999):

Example

```
var d = new Date();  
document.getElementById("demo").innerHTML = d.getMilliseconds();
```

The getDay() Method

The **getDay()** method returns the weekday of a date as a number (0-6):

Example

```
var d = new Date();  
document.getElementById("demo").innerHTML = d.getDay();
```

In JavaScript, the first day of the week (0) means "Sunday", even if some countries in the world consider the first day of the week to be "Monday"

You can use an array of names, and **getDay()** to return the weekday as a name:

Example

```
var d = new Date();
var days =
["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"];
document.getElementById("demo").innerHTML = days[d.getDay()];
```

UTC Date Methods

UTC date methods are used for working with UTC dates (Universal Time Zone dates):

Method	Description
getUTCDate()	Same as getDate(), but returns the UTC date
getUTCDay()	Same as getDay(), but returns the UTC day
getUTCFullYear()	Same as getFullYear(), but returns the UTC year
getUTCHours()	Same as getHours(), but returns the UTC hour
getUTCMilliseconds()	Same as getMilliseconds(), but returns the UTC milliseconds
getUTCMinutes()	Same as getMinutes(), but returns the UTC minutes

getUTCMonth()

Same as getMonth(), but returns the UTC month

getUTCSeconds()

Same as getSeconds(), but returns the UTC seconds

The setFullYear() Method

The **setFullYear()** method sets the year of a date object. In this example to 2020:

Example

```
<script>
var d = new Date();
d.setFullYear(2020);
document.getElementById("demo").innerHTML = d;
</script>
```

The setFullYear() method can **optionally** set month and day:

Example

```
<script>
var d = new Date();
d.setFullYear(2020, 11, 3);
document.getElementById("demo").innerHTML = d;
</script>
```

The setMonth() Method

The **setMonth()** method sets the month of a date object (0-11):

Example

```
<script>
var d = new Date();
d.setMonth(11);
```

```
document.getElementById("demo").innerHTML = d;  
</script>
```

The setDate() Method

The **setDate()** method sets the day of a date object (1-31):

Example

```
<script>  
var d = new Date();  
d.setDate(20);  
document.getElementById("demo").innerHTML = d;  
</script>
```

The setDate() method can also be used to **add days** to a date:

Example

```
<script>  
var d = new Date();  
d.setDate(d.getDate() + 50);  
document.getElementById("demo").innerHTML = d;  
</script>
```

If adding days, shifts the month or year, the changes are handled automatically by the Date object.

The setHours() Method

The **setHours()** method sets the hours of a date object (0-23):

Example

```
<script>  
var d = new Date();  
d.setHours(22);  
document.getElementById("demo").innerHTML = d;  
</script>
```

The setMinutes() Method

The **setMinutes()** method sets the minutes of a date object (0-59):

Example

```
<script>
var d = new Date();
d.setMinutes(30);
document.getElementById("demo").innerHTML = d;
</script>
```

The setSeconds() Method

The **setSeconds()** method sets the seconds of a date object (0-59):

Example

```
<script>
var d = new Date();
d.setSeconds(30);
document.getElementById("demo").innerHTML = d;
</script>
```

Compare Dates

Dates can easily be compared.

The following example compares today's date with January 14, 2100:

Example

```
var today, someday, text;
today = new Date();
someday = new Date();
someday.setFullYear(2100, 0, 14);

if (someday > today) {
    text = "Today is before January 14, 2100.";
} else {
    text = "Today is after January 14, 2100.";
```

```
}  
document.getElementById("demo").innerHTML = text;
```

JavaScript counts months from 0 to 11. January is 0. December is 11.

JavaScript Math Object

The JavaScript Math object allows you to perform mathematical tasks on numbers.

Example

```
Math.PI;           // returns 3.141592653589793
```

Math.round()

Math.round(x) returns the value of x rounded to its nearest integer:

Example

```
Math.round(4.7);    // returns 5  
Math.round(4.4);    // returns 4
```

Math.pow()

Math.pow(x, y) returns the value of x to the power of y:

Example

```
Math.pow(8, 2);      // returns 64
```

Math.sqrt()

Math.sqrt(x) returns the square root of x:

Example

```
Math.sqrt(64);       // returns 8
```

Math.abs()

Math.abs(x) returns the absolute (positive) value of x:

Example

```
Math.abs(-4.7);    // returns 4.7
```

Math.ceil()

Math.ceil(x) returns the value of x rounded **up** to its nearest integer:

Example

```
Math.ceil(4.4);    // returns 5
```

Math.floor()

Math.floor(x) returns the value of x rounded **down** to its nearest integer:

Example

```
Math.floor(4.7);    // returns 4
```

Math.sin()

Math.sin(x) returns the sine (a value between -1 and 1) of the angle x (given in radians).

If you want to use degrees instead of radians, you have to convert degrees to radians:

Angle in radians = Angle in degrees x PI / 180.

Example

```
Math.sin(90 * Math.PI / 180);    // returns 1 (the sine of 90 degrees)
```

Math.cos()

`Math.cos(x)` returns the cosine (a value between -1 and 1) of the angle `x` (given in radians).

If you want to use degrees instead of radians, you have to convert degrees to radians:

Angle in radians = Angle in degrees x $\text{PI} / 180$.

Example

```
Math.cos(0 * Math.PI / 180);    // returns 1 (the cos of 0 degrees)
```

Math.min() and Math.max()

`Math.min()` and `Math.max()` can be used to find the lowest or highest value in a list of arguments:

Example

```
Math.min(0, 150, 30, 20, -8, -200); // returns -200
```

Example

```
Math.max(0, 150, 30, 20, -8, -200); // returns 150
```

Math.random()

`Math.random()` returns a random number between 0 (inclusive), and 1 (exclusive):

Example

```
Math.random();    // returns a random number
```

Math Properties (Constants)

JavaScript provides 8 mathematical constants that can be accessed with the `Math` object:

Example

```
Math.E          // returns Euler's number
Math.PI         // returns PI
Math.SQRT2      // returns the square root of 2
Math.SQRT1_2    // returns the square root of 1/2
Math.LN2        // returns the natural logarithm of 2
Math.LN10       // returns the natural logarithm of 10
Math.LOG2E      // returns base 2 logarithm of E
Math.LOG10E     // returns base 10 logarithm of E
```

Math Constructor

Unlike other global objects, the Math object has no constructor. Methods and properties are static.

All methods and properties (constants) can be used without creating a Math object first.

Math Object Methods

Method	Description
abs(x)	Returns the absolute value of x
acos(x)	Returns the arccosine of x, in radians
asin(x)	Returns the arcsine of x, in radians
atan(x)	Returns the arctangent of x as a numeric value between $-\pi/2$ and $\pi/2$

<code>atan2(y, x)</code>	Returns the arctangent of the quotient of its arguments
<code>ceil(x)</code>	Returns the value of x rounded up to its nearest integer
<code>cos(x)</code>	Returns the cosine of x (x is in radians)
<code>exp(x)</code>	Returns the value of E^x
<code>floor(x)</code>	Returns the value of x rounded down to its nearest integer
<code>log(x)</code>	Returns the natural logarithm (base E) of x
<code>max(x, y, z, ..., n)</code>	Returns the number with the highest value
<code>min(x, y, z, ..., n)</code>	Returns the number with the lowest value
<code>pow(x, y)</code>	Returns the value of x to the power of y
<code>random()</code>	Returns a random number between 0 and 1
<code>round(x)</code>	Returns the value of x rounded to its nearest integer

`sin(x)`

Returns the sine of x (x is in radians)

`sqrt(x)`

Returns the square root of x

`tan(x)`

Returns the tangent of an angle